

IDE Plug-ins for Secure Coding

1. The Shift-Left Principle: Why Timing Is Everything

The shift-left principle states that finding and fixing security vulnerabilities at the earliest possible stage in the development lifecycle produces the highest return on security investment. IBM's Cost of a Data Breach Report 2024 quantifies what security professionals have known intuitively: a vulnerability caught and fixed by a developer during coding costs minutes. The same vulnerability found in a post-deployment penetration test costs days of remediation, potential downtime, and the risk that it was exploited in the window between deployment and discovery. Found after a breach, it triggers incident response, regulatory notification, and potentially litigation.

IDE security plug-ins exist because the IDE, the environment where developers spend their time writing code, is the earliest possible intervention point in the development pipeline. Catching a SQL injection pattern, a hardcoded credential, or a vulnerable library import before the developer commits the code is qualitatively different from catching the same issue in a CI/CD pipeline scan, a DAST tool, or a penetration test. The developer is still thinking about that specific code. No other system yet depends on the vulnerable implementation. Fixing it is trivial. This is why shift-left is a foundational concept for the exam.

2. The Four Primary IDE Security Tools

Tool	Vendor	Type	Key Strength	Notable 2024 Change
GitHub Copilot	GitHub / Microsoft	AI coding assistant with security features	Breadth of training data; secure code suggestions during completion	Added real-time vulnerability prevention flags in 2024
Amazon Q Developer	Amazon Web Services	AI coding assistant with security scanning	AWS-specific anti-patterns; full-file context scanning	Replaced AWS CodeWhisperer in 2024; expanded capabilities
Snyk Code	Snyk	Purpose-built security scanner	OWASP Top 10 accuracy; dependency vulnerability detection; CWE references	Consistently strong false positive rate; framework-specific rules
SonarLint	SonarSource	Open-source security analysis	Free; local analysis without cloud transmission; broad language support	Open-source companion to SonarQube; no server required

3. GitHub Copilot: Security in the Most Widely Deployed AI Coding Tool

GitHub Copilot is the most widely used AI coding assistant in the world, integrated with VS Code, Visual Studio, JetBrains, and other editors. In 2024, GitHub added real-time vulnerability prevention features that flag insecure code patterns as developers type rather than waiting for an explicit scan. Copilot can detect hardcoded credentials, SQL injection patterns, path traversal vulnerabilities, and other common security issues, and it can suggest secure alternatives inline.

The breadth of Copilot's training data, which includes enormous amounts of code from public GitHub repositories, is a double-edged characteristic. It has seen many examples of both vulnerable and secure implementations of common patterns, making its security suggestions grounded in real-world usage. However, this same training data includes a large volume of historically insecure code, which is why context-aware analysis rather than pure pattern matching is essential to Copilot's security feature quality.

4. Amazon Q Developer and Snyk Code

Amazon rebranded AWS CodeWhisperer as Amazon Q Developer in 2024, with expanded capabilities including real-time code suggestions, security scanning, and the ability to explain code and answer development questions. Amazon Q Developer's security scanning capability runs against the full file being edited, not just the current line, which gives it better context for detecting vulnerabilities that span multiple functions. For AWS-centric teams, it has specific advantages in detecting security issues with AWS SDK usage, IAM policy configurations, and infrastructure-as-code templates, catching AWS-specific anti-patterns that general-purpose tools would not be trained to recognize.

Snyk Code is purpose-built for developer security from the ground up, integrating with VS Code, IntelliJ IDEA, Eclipse, and other editors. It focuses specifically on identifying security vulnerabilities in real time using a combination of AI analysis and predefined security rules. Its particular strength is the OWASP Top 10 v2021 coverage, with high accuracy on SQL injection, XSS, broken access control, insecure deserialization, and other major vulnerability classes. Snyk Code provides detailed explanations referencing the relevant CWE identifier and remediation guidance, making it an education tool as well as a detection tool.

Exam Context: Amazon Q Developer, which replaced CodeWhisperer in 2024, and Snyk Code's purpose-built security focus are both testable distinctions. Know that Amazon Q Developer is the 2024 name for the AWS tool and that Snyk Code is not a general AI coding assistant but a security-specific scanner.

5. SonarLint: Open-Source Security for Broader Adoption

SonarLint is developed by SonarSource as the open-source IDE companion to SonarQube, a widely deployed enterprise code quality and security platform. SonarLint is available for VS Code, IntelliJ, Eclipse, and Visual Studio, and it performs real-time security analysis locally without requiring a server connection. This makes it practical for individual developers and teams that do not have an enterprise SonarQube deployment.

SonarLint's local operation mode is significant from a data handling perspective. While GitHub Copilot, Amazon Q Developer, and Snyk Code transmit code to cloud infrastructure for analysis and generation,

SonarLint can perform its analysis entirely on the local machine. For developers working on security-sensitive code, proprietary algorithms, or regulated data handling logic, the ability to receive security feedback without cloud transmission is a meaningful architectural distinction.

6. What These Tools Detect: Core Vulnerability Classes

Vulnerability Class	OWASP Top 10 v2021 Category	How IDE Plug-ins Detect It
SQL Injection	A03:2021 Injection	Detects string concatenation into database queries without parameterization; framework-aware to reduce false positives
Cross-Site Scripting (XSS)	A03:2021 Injection	Flags user-controlled content rendered in HTML without escaping; understands template engine contexts
Hardcoded credentials	A07:2021 Identification and Authentication Failures	Pattern matching on API key formats, passwords, tokens in source code; highest ROI detection
Insecure deserialization	A08:2021 Software and Data Integrity Failures	Detects deserialization of untrusted input without validation
Path traversal	A01:2021 Broken Access Control	Flags user input influencing file system paths without sanitization
Vulnerable dependencies	A06:2021 Vulnerable and Outdated Components	Checks imported library versions against CVE databases at import time (Snyk excels here)
Weak cryptography	A02:2021 Cryptographic Failures	Detects use of deprecated algorithms (MD5, SHA-1, DES) or incorrect cryptographic usage

7. Hardcoded Credentials: The Highest-ROI Detection

Of all vulnerability classes that IDE security plug-ins detect, hardcoded credentials provide arguably the highest security return. Credentials embedded in source code are among the most consistently exploited weaknesses across organizations of all sizes, because the compromise pathway is highly automatable and the exposure is often broader than the developer realizes.

When a developer hardcodes a credential in source code, the following cascade typically follows: the credential enters version control history and remains there even if the file is later corrected; it propagates to every developer who clones the repository; it gets included in build artifacts and container images; and it is frequently discovered by automated scanners continuously monitoring public repositories for credential patterns. GitGuardian's 2023 study found more than twelve million secrets exposed in public GitHub repositories during that year. Catching credentials before commit, at the IDE layer, breaks this cascade at its first step.

Real-World Incident: API Key Exposure via TypeScript Constant (Composite of documented 2023-2024 pattern): A developer at a cloud services company refactored authentication code and moved an API key from a configuration file format her IDE security plug-in scanned to a TypeScript constant in a utility module. The plug-in credential detection pattern did not cover that specific key format in that specific file

context. No flag appeared. She committed the code. Eleven minutes after it was pushed to a public GitHub repository, an automated credential scanner had found the key and it was being used to spin up cloud compute resources on the company account. The plug-in had been working correctly throughout the project. It simply did not cover that specific credential format in that context. This illustrates the false clearance limitation: tool coverage gaps do not announce themselves.

8. Context-Aware Analysis vs. Naive Pattern Matching

The quality gap between good IDE AI security tools and basic static analysis comes down to context awareness. A naive pattern matching tool might flag every database query containing a variable as a potential SQL injection risk. A context-aware AI understands that if the code is using Django's ORM, SQLAlchemy with parameterized queries, or Hibernate, the same code pattern is not a vulnerability because the framework handles parameterization safely.

GitHub Copilot understands which frameworks are in use by examining imports and project configuration. Snyk Code has framework-specific rules for Django, Spring, Express, and other common frameworks. This context-awareness directly determines the tool's false positive rate, which in turn determines whether developers trust the tool enough to pay attention to its flags. A tool with high false positives trains developers to ignore all flags, eliminating the tool's security value regardless of its technical capability. The exam tests the relationship between false positive rate, developer trust, and operational security effectiveness.

9. The False Positive Problem: How Tools Fail Operationally

False positives are not just an annoyance in IDE security tools. They are the primary mechanism through which these tools fail operationally. When a plug-in flags secure code as vulnerable, one of three outcomes follows: the developer learns the pattern and dismisses it correctly, which requires time and effort; the developer wastes time investigating something safe, eroding trust in the tool; or, most dangerously, the developer starts dismissing all flags without review because the false positive rate has made the tool feel like noise.

When a developer has learned to ignore the plug-in's warnings, the plug-in provides zero security value regardless of its technical capability. This failure mode is silent: the tool continues running, continues reporting statistics, and appears to be providing security coverage while actually providing none. Managing false positive rates is a shared responsibility between the security team configuring the tool's sensitivity settings and the development team communicating which flag categories are most meaningful to them.

10. Dependency Vulnerability Detection: Supply Chain at Import Time

When a developer adds a library to their project, there is a window of opportunity to check whether that specific version has known vulnerabilities before it becomes part of the codebase. Snyk Code is particularly strong at this: as developers type an import statement or add a package to a dependencies file, Snyk checks that library version against its vulnerability database and flags any known CVEs with severity ratings and recommended remediation.

Amazon Q Developer does similar checking for AWS SDK versions and common Node.js and Python packages. This capability addresses the software supply chain risk at its earliest intervention point, before the vulnerable dependency is committed, before it is deployed, and before it becomes an exploitation pathway. The OWASP Top 10 v2021 A06:2021 Vulnerable and Outdated Components category documents how widespread and exploited this attack class is; detecting it at the IDE layer is where the effort has maximum leverage.

11. What IDE Plug-ins Cannot Catch

Building an effective defense strategy requires understanding what these tools miss, not just what they detect. IDE security plug-ins have characteristic coverage gaps that determine where other security layers must compensate.

- Logic flaws requiring business context: a feature that allows users to access other users data is functionally correct code. No security plug-in flags it because it is not a code pattern violation; it is an authorization design error that requires understanding of business requirements to identify.
- Race conditions and timing vulnerabilities: these only manifest under specific runtime conditions and concurrent execution patterns that cannot be detected from static code analysis.
- Component interaction vulnerabilities: when individually correct components interact in ways that create security issues, static IDE analysis of each component finds nothing wrong.
- Coverage gaps on specific patterns: as the opening scenario illustrates, a tool working correctly on all documented patterns will still miss patterns not in its coverage set. The false clearance does not announce itself.
- **False security signal risk:** developers who see consistent clean results from IDE plug-ins may develop over-confidence in the security of code that has only been checked against the tool's known patterns.

12. IDE Plug-ins in the Defense-in-Depth Pipeline

IDE plug-ins are the first gate in the development security pipeline, not the only gate. Code that passes IDE checks should face additional analysis at each subsequent stage: CI/CD pipeline scanning with tools like GitHub Advanced Security, Snyk's CI integration, Semgrep, or Checkmarx for full-codebase analysis with more time and context than the IDE has available; container scanning tools like Trivy examining built artifacts; dynamic analysis testing running code; and penetration testing examining deployed systems.

These layers are complementary rather than redundant. The IDE layer catches issues while the developer is still thinking about that code. The pipeline layer catches what the IDE layer missed with comprehensive analysis. The dynamic layer catches issues that only appear at runtime. The penetration testing layer catches issues that require human creativity to find. Defense-in-depth applied to the development security pipeline means each layer catches what the previous layer could not, rather than duplicating the same checks at each stage.

13. Data Handling: Code Transmission to Cloud Infrastructure

When using GitHub Copilot, Amazon Q Developer, or any cloud-hosted IDE AI tool, code from the project is transmitted to the vendor's cloud infrastructure for analysis and generation. For most development work this is acceptable under standard enterprise agreements. For code implementing security controls, handling personally identifiable information, representing proprietary business logic in a regulated industry, or containing sensitive security research, this transmission requires explicit data classification consideration.

Amazon Q Developer and GitHub Copilot both offer enterprise agreements with specific data handling commitments, including options to disable training data use. SonarLint's local operation mode performs all analysis without cloud transmission. For security-sensitive codebases, understanding each tool's data handling mode and configuring it to match data classification requirements is part of responsible deployment, not an optional configuration preference.

Key Terms Glossary

Term	Definition
Shift-left security	The principle that finding and fixing vulnerabilities during development is significantly cheaper and more effective than finding them later; IDE plug-ins are the primary tool implementing this principle.
Amazon Q Developer	AWS AI coding assistant that replaced CodeWhisperer in 2024; includes real-time security scanning with full-file context; particularly strong on AWS-specific anti-patterns.
Snyk Code	Purpose-built developer security tool integrated in the IDE; focused on OWASP Top 10 v2021 coverage with CWE references and framework-specific rules; strong dependency vulnerability detection.
SonarLint	Open-source IDE security plug-in from SonarSource; companion to SonarQube; supports local analysis without cloud transmission; no server required.
False positive	A security tool flag on code that is not actually vulnerable; high false positive rates cause developers to ignore all flags, eliminating the tool's security value.
Context-aware analysis	IDE security analysis that understands the frameworks and patterns in use, not just the raw code text; reduces false positives by recognizing when a framework handles the security concern.
False clearance	The dangerous scenario where a security tool gives a clean result on code that contains a vulnerability outside its coverage pattern; the tool was working correctly; it just did not cover that pattern.
CWE	Common Weakness Enumeration; a community-maintained classification of software security weaknesses used by Snyk Code and other tools to explain why a flagged pattern is a vulnerability.

Quick Revision Checklist

- Shift-left principle: fixing vulnerabilities during coding is far cheaper than fixing them post-deployment; IDE plug-ins are the earliest possible security intervention point.

- IBM Cost of a Data Breach Report 2024 supports the shift-left economic argument with cost data across development lifecycle stages.
- GitHub Copilot: most widely deployed; added real-time vulnerability prevention in 2024; context-aware security suggestions during code completion.
- Amazon Q Developer: replaced AWS CodeWhisperer in 2024; full-file context scanning; strong on AWS SDK, IAM policy, and infrastructure-as-code security issues.
- Snyk Code: purpose-built security scanner; OWASP Top 10 v2021 coverage; CWE references; strong dependency vulnerability detection at import time.
- SonarLint: open-source; local analysis without cloud transmission; companion to SonarQube; no server required; broad language support.
- Hardcoded credentials: highest-ROI detection; cascade risk through version control, repository clones, build artifacts, and container images.
- Context-aware analysis reduces false positives by recognizing framework-handled security patterns (Django ORM, SQLAlchemy, Hibernate).
- False positives cause operational failure: developers who learn to ignore flags receive zero security value from the tool regardless of its technical capability.
- IDE plug-ins cannot catch logic flaws requiring business context, race conditions, or component interaction vulnerabilities.
- IDE plug-ins are the first gate, not the only gate; defense-in-depth requires CI/CD scanning, container scanning, DAST, and penetration testing as additional layers.
- Cloud-hosted tools transmit code to vendor infrastructure; SonarLint local mode and enterprise agreements with training-data-use restrictions address data sensitivity requirements.

10 Exam-Based MCQs - IDE Plug-ins for Secure Coding

These questions reflect the scenario framing and distractor style used in the CompTIA SecAI+ exam. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A fintech organization has experienced three SQL injection vulnerabilities in production over the past year. All three were found by customers rather than by internal testing. The security team is proposing a new control to shift vulnerability detection as early in the development lifecycle as possible. An organization wants to reduce the number of SQL injection vulnerabilities reaching production by implementing a control at the development stage. Which control BEST aligns with the shift-left security principle?

- A) Deploying a web application firewall (WAF) in front of the production database
- B) Running a DAST scan against the staging environment before each production deployment
- C) Configuring IDE security plug-ins to flag SQL injection patterns as developers write code
- D) Requiring developers to attend SQL injection awareness training annually

Answer: C **Explanation:** C is correct because IDE security plug-ins operate at the earliest possible point in the development lifecycle: the moment the developer is writing the code. This is the definition of shift-left: moving the security intervention as early as possible. A is wrong: a WAF operates at runtime in production; this is the opposite of shift-left. B is wrong: DAST against staging is a later-stage control than IDE detection; shift-left favors the earliest intervention. D is wrong: annual training is an awareness control, not a

technical detection control; it does not directly catch specific vulnerabilities as code is written.

Q2. A developer moves an API key from a configuration file that the IDE security plug-in scans to a TypeScript utility module. The plug-in does not flag it. Eleven minutes after pushing to a public repository, the key is used by an attacker to spin up cloud resources. Which concept does this scenario BEST illustrate?

- A) A false positive: the plug-in incorrectly flagged the code as secure
- B) A false clearance: the plug-in gave a clean result on a pattern outside its coverage set
- C) A hallucination: the plug-in generated incorrect security guidance for the file
- D) A prompt injection: the attacker embedded instructions in the code that disabled the plug-in

Answer: B **Explanation:** B is correct because the plug-in was working correctly on all patterns it covers; it simply did not cover the specific credential format in the specific file context. The clean result was not an error in the tool's logic for covered patterns; it was a gap in the tool's coverage scope. This is a false clearance: the developer received an implicit signal that the code was secure when it was not. A is wrong: a false positive is when a tool flags secure code as vulnerable; this scenario is the opposite. C is wrong: hallucination is an LLM concept about generating incorrect content; IDE security plug-ins are not LLMs generating text. D is wrong: prompt injection requires malicious instructions embedded in content the AI processes; this is a coverage gap, not an injection attack.

Q3. Scenario: A security team is reviewing their IDE security program. Developers have stopped reviewing plug-in warnings. Review of the plug-in logs shows it has been flagging Django ORM queries as SQL injection vulnerabilities at a rate of approximately forty false positives per developer per day. A security team notices that developers have stopped acting on IDE security plug-in warnings even when the flags are legitimate. Investigation reveals that the plug-in has been flagging parameterized Django ORM queries as SQL injection vulnerabilities. Which root cause BEST explains the security failure?

- A) The developers have been compromised by an insider threat that is suppressing security alerts
- B) High false positives from a tool that lacks context-aware analysis have caused developers to dismiss all flags
- C) The IDE plug-in's threat intelligence feed has not been updated in six months
- D) The developers require SQL injection training to understand which flags are legitimate

Answer: B **Explanation:** B is correct because Django ORM queries with parameterization are not SQL injection vulnerabilities; a tool flagging them is producing false positives. When developers repeatedly see that a tool's flags are wrong, they learn to dismiss all flags, including legitimate ones. The root cause is the tool's lack of framework context-awareness combined with the resulting developer trust erosion. A is wrong: the scenario describes a systematic false positive problem, not insider threat behavior. C is wrong: threat intelligence feed staleness affects vulnerability database currency, not framework context awareness for known code patterns like Django ORM. D is wrong: the developers may be correct that the flags are false positives; the problem is the tool, not developer understanding.

Q4. Which AWS AI security tool replaced CodeWhisperer in 2024 and provides full-file context scanning for security vulnerabilities?

- A) AWS Security Hub AI Assistant
- B) Amazon Q Developer
- C) Amazon GuardDuty Copilot
- D) AWS Trusted Advisor AI

Answer: B **Explanation:** B is correct because Amazon Q Developer is the 2024 rebrand and expansion of AWS CodeWhisperer, adding broader capabilities including security scanning with full-file context and natural-language code assistance. A is wrong: AWS Security Hub AI Assistant is not the name of this product. C is wrong: Amazon GuardDuty is a threat detection service; it does not have a Copilot product that replaced CodeWhisperer. D is wrong: AWS Trusted Advisor is a cost and configuration optimization tool, not an IDE AI security product.

Q5. A developer at a healthcare company is writing code that handles patient PII and wants IDE security analysis without transmitting the code to external cloud infrastructure. Which tool BEST meets this requirement?

- A) GitHub Copilot with enterprise agreement
- B) Amazon Q Developer with training data use disabled
- C) SonarLint in local analysis mode
- D) Snyk Code with the HIPAA compliance configuration enabled

Answer: C **Explanation:** C is correct because SonarLint is specifically noted for its ability to perform security analysis locally without requiring cloud transmission, making it directly applicable to sensitive code that should not leave the local development environment. A is wrong: GitHub Copilot with an enterprise agreement provides data handling controls including disabling training data use, but code is still transmitted to GitHub's infrastructure for analysis and suggestions. B is wrong: disabling training data use prevents the code from being used for model training but does not prevent transmission to Amazon's infrastructure for analysis. D is wrong: Snyk Code does not have a documented HIPAA compliance configuration that enables local-only analysis; it is a cloud-connected service.

Q6. According to OWASP Top 10 v2021, which vulnerability category does Snyk Code's dependency vulnerability detection at import time MOST directly address?

- A) A03:2021 Injection
- B) A01:2021 Broken Access Control
- C) A06:2021 Vulnerable and Outdated Components
- D) A07:2021 Identification and Authentication Failures

Answer: C **Explanation:** C is correct because A06:2021 Vulnerable and Outdated Components covers the use of libraries and dependencies with known CVEs. Snyk Code's capability of checking imported library versions against its vulnerability database at the moment of import directly addresses this category by catching vulnerable dependencies before they are committed. A is wrong: A03:2021 Injection covers SQL injection, XSS, and similar code injection vulnerabilities, not dependency vulnerability detection. B is wrong:

A01:2021 Broken Access Control covers authorization design flaws, not dependency security. D is wrong: A07:2021 covers authentication weaknesses, not dependency vulnerabilities.

Q7. IDE security plug-ins are BEST characterized as which layer in the development security pipeline?

- A) The only layer needed if the plug-in is properly configured with up-to-date signatures
- B) The first gate, detecting issues as code is written, complemented by CI/CD scanning, container scanning, and DAST in later stages
- C) A replacement for penetration testing because they analyze all code paths automatically
- D) A post-deployment monitoring tool that detects vulnerabilities in running applications

Answer: B **Explanation:** B is correct because IDE plug-ins are the first gate in the development security pipeline, not the only gate. They catch issues at the earliest stage; subsequent layers (CI/CD scanning, container scanning, DAST, penetration testing) catch what the IDE layer missed with more context, more time, and different analysis approaches. A is wrong: IDE plug-ins have inherent coverage gaps and cannot be the only security layer; defense-in-depth requires multiple complementary layers. C is wrong: IDE plug-ins use static analysis on code as written; penetration testing involves active exploitation attempts against running systems, which catches a fundamentally different class of vulnerabilities. D is wrong: IDE plug-ins analyze code during development, not running applications post-deployment.

Q8. Which vulnerability class CANNOT be reliably detected by IDE security plug-ins because it requires understanding of business requirements beyond what can be inferred from code patterns?

- A) SQL injection in string-concatenated database queries
- B) Hardcoded API keys and credentials in source code
- C) Broken access control where a feature allows users to access other users' data due to an authorization design error
- D) Insecure cryptography using deprecated algorithms like MD5

Answer: C **Explanation:** C is correct because an authorization design error, where a feature is functionally correct code that allows inappropriate data access by design rather than by code error, cannot be detected by pattern matching or AI code analysis. The code is syntactically correct, uses no inherently vulnerable patterns, and appears to do what it is designed to do. Identifying the authorization flaw requires understanding the intended business rule that the code violates. A is wrong: string-concatenated database queries are a code pattern that IDE tools reliably detect. B is wrong: hardcoded credentials are one of the most reliably detected patterns across all IDE security tools. D is wrong: use of deprecated cryptographic algorithms is a code pattern that IDE tools detect through library import analysis and algorithm name matching.

Q9. The shift-left security principle is MOST directly supported by which economic finding from IBM's Cost of a Data Breach Report 2024?

- A) Organizations with AI-powered security operations reduce breach costs by an average of \$1.8 million

- B) Vulnerabilities found and fixed during development cost a fraction of what they cost to remediate post-deployment or post-breach
- C) The average cost of a data breach reached \$4.88 million in 2024, up from \$4.45 million in 2023
- D) Organizations that use zero trust architecture reduce breach costs significantly compared to those that do not

Answer: B **Explanation:** B is correct because the shift-left principle is an economic argument: finding vulnerabilities earlier in the development lifecycle costs less than finding them later. IBM's report provides the cost data that quantifies this gradient across development, testing, deployment, and breach stages. A is wrong: AI security operations reducing breach costs supports investment in AI security tools but does not directly support the shift-left principle about development lifecycle timing. C is wrong: the overall average breach cost statistic describes the severity of breaches generally but does not speak to the timing-cost relationship that shift-left addresses. D is wrong: zero trust architecture is an access control design principle, not a development lifecycle timing principle.

Q10. A developer using GitHub Copilot receives a code suggestion that implements user input validation using a pattern that appears secure. She accepts the suggestion without additional review. A subsequent code review by a security engineer identifies that the AI-suggested validation logic has a subtle bypass under a specific input format. Which principle does this scenario MOST directly illustrate?

- A) Prompt injection through the developer's query to Copilot
- B) The need for human validation of AI-generated code, especially for security-critical paths
- C) A supply chain attack through the GitHub Copilot model training pipeline
- D) False positive generation by GitHub Copilot's security analysis feature

Answer: B **Explanation:** B is correct because the scenario illustrates the red team perspective on IDE AI trust: AI code suggestions can introduce subtle vulnerabilities that appear secure but have edge-case bypasses. The mitigation is treating AI suggestions as a draft requiring human security review, not a final implementation. AI generates; practitioners validate. A is wrong: prompt injection requires malicious instructions embedded in content the AI processes; this is about trust in AI-generated code quality, not injection. C is wrong: supply chain attacks on model training pipelines are a real threat category but are not what this scenario describes, which is a quality limitation in AI code suggestion output. D is wrong: Copilot's security analysis feature flagging code as safe or unsafe is separate from the code it generates; the scenario describes a vulnerability in a Copilot-generated suggestion, not a false positive in its scanning output.